!! remind myself to add mouse functions !!
!! and keyboard interrupt !!
!! add div/mul !!
!! and use floats for screen, not ints and round !!
^^ this is not part of the docs, this is a reminder for myself!!

1000 storage spots on the macro version
100 on the micro version

Macro is good for regular computers and stuff like that
Micro is good for slower computers and things with not that much storage

You can run a micro program on macro but you cannot run a macro version on micro

If you are making code or publishing it, it would be good to specify if you are making it for micro or macro, think of macro and micro like separate coding languages for now.

For program storage space, it doesn't really matter too much but give as much as you can get but do try make the programs small, or at least make a smaller version of your program for micro users on like an esp32 for instance

m - (mov, e.g m source, destination)
p - (print)
a - (add, times)
s - (subtract, divide)
f - (function)
e - (end, see "f")
c - (call)
j - (jump)
i - (if)
l - (letter but you can pass with j to join)
t - (time. This includes things like waiting before running the next command and waiting then running a function)
d - run display (this is not “render”, it just tells the computer to use the display to display stuff so run this at the start of your code.)

!! ALERTS LOOK LIKE THIS !!

!! DO NOT USE INDENTATION! IT CONFUSES THE COMPUTER !!

Specify if an input value is a certain type like this
"String" #this is a string
(1234) #this is a number
x7 #this is a storage sector

Display info:

The display is written as 1920x1080px but can be scaled depending on the size of the monitor but will not change the numbers you input. So for instance on a 1920x1080px screen the numbers would line up with the pixels but on a 720x480px screen it would just size it down. This does lead to a little bit of stretching but it shouldn't be too bad and most displays are 1920x1080 anyways, or at least a similar ratio.

!! REMEMBER TO PUT ANY DISPLAY STUFF AT ALL INSIDE A FUNCTION LOOP OR IT WILL ONLY RENDER FOR ONE FRAME !!

!! BY THAT I MEAN USE THE JUMP COMMAND !!

The first 7 spots hold graphics stuff like this
x1 - X position 1
x2 - Y position 1
x3 - X position 2
x4 - Y position 2
x5 - color (in r,g,b so like (100,100,100) for grey) ← format as a string
x6 - shape (can be either "circle", "square", "triangle", "filledcircle", "filledsquare", "filledtriangle‘, "text")

Also with the shapes, circle, square and triangle, unless specified as filled it will have an outline of 1px

And x7 holds input using strings.

And x8 holds mouse X
And x9 holds mouse Y

For example if you pressed the up arrow it would change to "up".
Its very standard but if you are doing function keys it's just "F4" or so on

I'll add a list of key names at the end

With text you must specify instead of the normal way you do this:

x1 - X position
x2 - Y position
x3 - Text size (px)
x4 - Text
x5 - color
x6 - shape again to specify that it is text

And with triangles, you must do this:

x1 - X position (origin)
x2 - Y position (origin)
x3 - X position (top point)
x4 - Y position (top point)
x5 - Color
x6 - shape
And when you specify it is a triangle, the computer will save it to ram and wait till you call a triangle again to finish the rest of the parameters like this:

Parameters 2

x1 - X position (bottom point)
x2 - Y position (bottom point)
x3 - outline thickness (0 for none, good for non-filled triangles but measured in px)
x4 - none (just put the text “none” though.)
x5 - the color again
x6 - shape - make sure to pass this as the same type of triangle (triangle, filledtriangle) or it wont work :)

Also, to constantly render it, you need to run it through a loop using a function and a jump inside the function. This can cause flickering if not properly implemented but you can have it find how many commands it runs then wait for all the commands to finish.. Or basically any other way, doesn't really matter.

So this is how you’d do it properly

```
d
j "poop"

f "poop"
m (0) x1
m (0) x2
m (1920) x3
m (1080) x4
m "(255,255,255)" x5
m "circle" x6

m (0) x1
m (0) x2
```

```
m (1920) x3
m (1080) x4
m "(255,0,0)" x5
m "square" x6


j "poop"
e
```

m docs:

Use "m" for moving things

!! DO NOT USE SECTORS UP TO x7 FOR STORING THINGS, THEY WILL NOT SAVE AND BE SENT TO THE SCREEN !!

x10 is chill

Storage gets called from 0-100 with an x at the start

e.g…

```
m "hello", x10
m x10, x11
# this is a code comment
# this code moves hello to x11
# mov copies the first value to the second value but leaves the first value the same
m (0) x1
m (0) x2
m (100) x3
m (100) x4
m "(100,100,100)" x5
m "circle" x6
# this code creates a circle
# well only for one frame.. You'd need to put it in a loop
```

Storage to storage moving is allowed

e.g..

m x1, x5
all good

p docs:

just prints out the text or storage spot

e.g ..

p x23
#prints out what is inside x23
p "hello"
#prints hello

!! YOU DO NOT NEED TO MOVE YOUR TEXT TO A SECTOR BEFORE YOU PRINT IT !!
You can do it simply like
p "see? no sectors!"

a docs:

Format: "a (additive number) (storage sector)"
Format for timesing: “a t (number to times by) (storage sector)”

"a" adds to the specified value by replacing it with the original value but added up by an inputted number
And t does the same thing but with times

e.g:
a (1) x10
this will add 1 to the x10 number

You can also add sectors like
a x10 x11
and times
a t x10 x11
This would get the data from x10, add it to x11 then put it in x11

!! this will error if it is not a number !!
to solve this you can run a m command to make it a number like so:

m (1) x10
a (2) x10

Also, it will error if you try to do a number plus another number without any way to output it:

a (1) (2) ← this will not work

this will not give you 3

s docs:

s is the same formatting and whatnot as a but just subtracting

And you can use dividing here, too like

s d (divide number) (storage)

f docs:

f is for calling functions like so:

f "functionname"

!! END YOUR FUNCTIONS WITH "e" PLEASE!!!! !!

!! YOU CANNOT PUT SPACES IN THE FUNCTION NAME OR THE COMPUTER WILL GET CONFUSED !!

!! ALSO CREATE YOUR FUNCTIONS AT THE END OF THE CODE AND CALL THEM AT THE START OR IT SKIPS IT !!

So like
j "funktion"

f "funktion"
p "wont you take me to"
p "funcy town?"
e

e.g:

f "hello"
#i'll show you how to call this next

m x10 x11

To end the function, call "e" for end

f "noargs"

Some test code:

f "hello"
m x10 x11
e

If you run this function, it would move whatever you put in sector to x7

c docs:

you can call functions with c like this:

c hello

in this example, look up for reference, this would move x10 to x11

However, when you run c it will run the function then *take you back* to the code the c was run from, versus j which runs the function and stops afterwards if there are no more jumps or calls to do.

j docs:

Just like a call but does not take you back to the code and runs the function then stops.

d docs:

d calls the display and tells the computer to use the display

!! IF YOU ARE USING A DISPLAY MAKE SURE TO CALL THIS AT THE VERY TOP OF THE CODE, BEFORE ANY CODE COMMENTS OR ANYTHING !!

i docs:

you can compare using i:

format:
i x10 = x11 x12
^^ this will put either 0 or 1 in x12 based on the output of the statement

Different comparators:

= (.. is the same as..)
> (.. is bigger than..)
< (.. is smaller than..)
!= (.. is not..)
=! (.. is not..)

You can compare strings, numbers, and the contents of a sector.
It will output to the last argument but it will compare the first two:

i x10 = x11 x12

How it outputs is in 0s and 1s as numbers but you can compare strings and such others

You can also run functions if it is correct like this:
i x10 = x11:functionname:

It will jump back and continue like a call function after the function has been run

m (1) x13
m (1) x12
i x13 = x12 :banan:

f "banan"
p "it is in fact yes"
e

this is in fact yes

Also you probably can't do > on a string but you can do a = on a string and a !=

This means you can do

j "key"

f "key"
i x7 = "space" "print"

j "key"
e

f "print"
p "you pressed space!"

e

l docs:

l stands for letter and it’s pretty easy to use
l "text" (letter) x10 ← output

So if you wanted to find the 4th letter in the word "sausage" you would do this:

l "sausage" (4) x10

You can also find the letter in a sector like this:

l x10 (4) x11

If x10 was "sausage" then it would put s in x11

You can pass a "j" to join text together like this:

l j "hello " "world" x10
^^ this would join to be "hello world" and put it in x10
Pretty self explanatory so im not gonna bother explaining

!! NOT TO BE CONFUSED WITH "i", THIS IS "L"

t docs:

You can run t to wait and run a function like this:

t w (1) :function:

You cannot do

t w (1)

And expect it to work because it wont
You NEED to call a function..

It is measured in seconds btw
And you can do floats

When you run a function, it runs it like a call command and returns to the original line afterwards

Key names:

When you are not pressing a key, it will set x7 to the text "none". Just letting you know

Alphabet keys are just the name of the letters of the alphabet, e.g "H" is just H

It is not case sensitive, so if you press shift and another key it will come up with "shift a" in x7

But yeah if you press more than one key at a time x7 will show "a b c d"

Shift - "shift"
Control - "ctrl"
Alt - "alt"
Windows, Mac or home key etc if you can gain access to it - "logo"
^^^ do not rely on this for your program to work as some people may not have this ^^^
Space - "space"

ALSO all keys that are case sensitive like ` and / will ALWAYS be the not capitalized version of the key

Same goes for numbers

So instead of "?" it would be "/"

Likewise with , and . they're not <>

Caps Lock - "caps" ← also don't let your program rely on this

Tab - "tab"

Arrow keys:

Up - "up"

Down - "down"
Left - "left"

Right - "right"

Enter - "enter"

And then the rest is just whatever they are called but not capitalized.

And x8 and x9 are mouse x and mouse y